

MSDS 640 Capstone Project Report: Classifying and Predicting NBA Breakout Players using Machine Learning Models Supported by Statistical Analysis

By: Conor Eunson

Project Abstract

The goal of this project was to prove whether it was possible to predict whether NBA players would break out or not in a given season using their seasonal statistics. The project hypothesis was tested using a seasonal train, validation, test data split on a majority classifier, Logistic Regression, and XGBoost machine learning models. After tuning hyperparameters for each machine learning model, the results produced by the evaluation metrics accuracy, precision, recall, f1-score, and confusion matrix indicated XGBoost was the best performing model at predicting the target breakout. In order to confirm if the results from the best performing model had any meaningful impact, two statistical tests, which were the Paired t-test and McNemar's test, were conducted to determine if the XGBoost model had any statistical significance. XGBoost when compared to both the Baseline Logistic Regression and tuned Logistic Regression failed to show any statistical significance in its ability to predict breakout. XGBoost's inability to show any meaningful difference statistically led the project to conclude that the hypothesis is both unsupported and not usable for real world use without further modification, research, and testing.

Introduction and Problem Statement

Every year the NBA has a group of players that seem to suddenly improve significantly compared to their previous performance surprising both teams and fans of the NBA. While in some cases it can seem easy to predict when certain NBA players will breakout into being one of the best players in the league, other players can be more unpredictable taking six to eight seasons of development before they finally breakout into elite level players. However, through personal experience viewing professional basketball as well as reviewing NBA player research papers, as things currently stand it is extremely difficult for both NBA organizations and fans of the league to predict if a given player will breakout each season. This lack of consistency or objective statistical evidence causes teams around the league to give up on players too early with the belief that they will never breakout into one of the top tier NBA players. When one of these players the team believes is a "bust" is forced to leave, then breaks out into being one of the fifty best players in the league the following season results in team executives getting fired, team revenue loss, and can cost the team their chance to win their first ever championship in franchise history. With this background in mind, the goal of this project was to solve this problem by utilizing NBA statistical player data from previous seasons, along with machine learning models to accurately classify and predict when NBA players are going breakout.

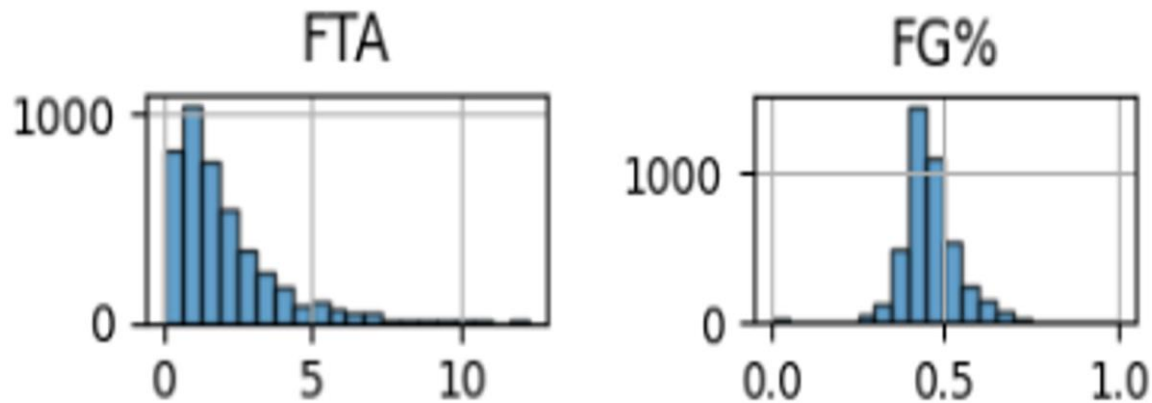
Going into this project report, there are two key research questions along with the overall hypothesis that are important to highlight during the early stages of this paper. The first research question is, how do we want to quantify or classify a “Breakout” player? The second research question is by utilizing NBA player statistics, are we able to find a way to accurately predict whether they will breakout in a given season? These two research questions served as the foundation for the development of this project’s hypothesis. The hypothesis for the project is NBA players that see an increase of 20% or more to their Points Per Game (PTS) and an increase of 2 or more Win Shares (WS) from one season to the next will breakout. The threshold values used within this hypothesis are not arbitrary values and will be thoroughly discussed during the feature engineering section within this report.

Data, Data Analysis, and Feature Engineering

All the data that was collected and used throughout the course of this project was from the website basketball-reference and the link for this website can be found in the bibliography/table of contents section at the end of this paper. Basketball-reference is a free to use website that provides historical basketball data dating from the 1940’s up to the most recent 2025-26 NBA season at the time this report was written. There is a large variety of basketball data from individual player statistics, team historical records, and NBA entire season outcomes just to name a few. The data collection process from basketball-reference was overall quite a simple process. The first decision was to decide how many seasons’ worth of player data would be taken. A total of 11 NBA seasons worth of player data from the timeframe 2015-16 to 2025-26 was used for the purposes of this project. Basketball-reference allows anyone to download this data as an excel file for each of the individual seasons. In order to have a wide variety of columns or features to analyze and train machine learning models, the decision to use advanced basketball statistics was made. These advanced statistics had to be downloaded as their own separate individual excel files and then merged with the players other seasonal statistics for all 11 seasons. The final step to create the projects’ finalized data csv file was to combine each of the seasons’ player data into one single file containing all the seasons from each individual season excel file. This was done manually to ensure no data errors or loss during the transfer of the data. The final count for records within the finalized dataset was 5,968 and a total of 53 columns or features.

Conducting exploratory data analysis before adjusting the dataset significantly was a necessary and important step in understanding feature trends within the dataset. Checking the datatypes present within the dataset, it was not surprising to observe that a large majority of features in the dataset were numeric with a limited number of categorical features like player name (Player) and team initials (Team). When plotting and observing the histogram distributions of the features, they tended to follow one of two observable patterns. The first group would follow a left leaning distribution where a majority of values were plotted at the lowest values sharply declining as the range of values increased. The rest of the columns followed a normal or standard distribution with most of the values being found in the middle of the plot with the

number of values declining as the data increased or decreased. Examples of histograms for each of these two groups of histogram distributions will be labeled and can be found below. Left figure is left leaning distribution with the figure on the right illustrating normal distribution.



Before the creation of the target variable can be done through feature engineering it is imperative that time is spent on properly preprocessing the dataset. Starting with handling missing values within the dataset, a function was utilized in order to streamline the process of observing the total amount of missing values per feature. The outlier column was Awards with roughly 90% of its values missing making dropping the feature easy to justify. Every column except for Awards had a very limited number of missing values, around 1 to 7%, that needed to be dealt with. When examining these missing values closely, a discovery was made that made it quite simple to replace these values. When looking closer into the columns that had these missing values, a common pattern amongst them emerged. Using Field Goal % (FG%) column as an example, this column is calculated by taking a player's Field Goals Made (FG) and dividing it by their number of Field Goals Attempted (FGA) to get that seasons' FG%. When examining rows where FG% was null, that player's number of FG and FGA equaled zero. This means that the FG% was not calculated because this player never took this type of shot the entire season. The pattern was observed with all the other features with missing values, making it easy to justify setting the missing values for all these columns equal to zero. The dataset after the creation of target variable breakout had a clear majority class imbalance of 93/7. After evaluating all options to handle this imbalance, model parameters of `class_weight` and `scale` were decided as the preferred method to ensure data integrity. Regarding scaling or standardizing the data due to all features falling within a tight range of values from 0 to 40, it was deemed unnecessary to use popular methods like standard scalar to normalize the data close together. Establishing the correct data splitting strategy was one of the vital parts of the project. The final decision made was to use a looping seasonal train, validation, and test splits to ensure the machine learning models were following the sequential time series order present within that dataset. An in-depth discussion on data splitting and details on how it is structured can be found under the methodology section.

This section will highlight the feature engineering and selection done to improve the datasets' ability to predict the target breakout. The season (SEASON) feature represents the years that league season took place in. However, its original format was 2015-16 made it a categorical feature which limited the features' ability to impact the machine model using seasonality and time. The values for this feature were adjusted to be numeric values of the year each season started meaning that in this example the new value for season would just be 2015. The position (Pos) column signifies the basketball role each player has with the NBA. This is a categorical feature has 5 total groupings for the five roles on the court, which are Point Guard (PG), Shooting Guard (SG), Small Forward (SF), Power Forward (PF), and Center (C). These text values without any changes are not useable or useful features in terms of helping with the binary classification and prediction process. Using `get_dummies`, documentation can be found below, each position was able to be categorically encoded into numeric values of zero, which indicates that the player does not play that position and one meaning the opposite.

Creation of the target variable occurred during this stage of the project and it is worth the time to explain step by step how the target or breakout feature was created. Ideally the goal would have been to find research studies from institutions on the topic of NBA players breaking out to utilize the features and thresholds to objectively test for this breakout factor. However, no study or report on this topic was found during research making it difficult to define breakout and necessitating the need to define these thresholds by some other method. The NBA every year hands out a group of awards to players around the league to celebrate a player's individual achievements each season. One of these awards is the Most Improved Player Award (MIP) which is voted on and given to the player who the NBA believes improved the most that season compared to their previous year's performance. After taking a closer look, it was discovered that basketball-reference had a page for seasonal awards providing the top five players voted for MIP and their statistics that season. This MIP players data served as the catalyst for objectively defining what a set of feature conditions signify a NBA player breaking out. To ensure a decent sample size for observing trends in MIP players statistics, 7 seasons worth of excel data was downloaded. The goal of this process was to observe and analyze the differences in values between these players' statistics the year before and the year they were voted for or won the MIP award. The two methods that were used evaluate the difference in player statistics were percentage differential or percentage change for the more simplified accounting stats, like PTS and for calculated statistics with complex formulas, like Box Plus/Minus (BPM) numerical difference in values was used.

$$\text{Percentage Change} = \frac{\Delta V}{|V_1|} \times 100$$

$$= \frac{(V_2 - V_1)}{|V_1|} \times 100$$

Actual/Numerical Change:

New – Old = Difference

The above figure contains the two formulas that were used to calculate the differences in player statistics. On the left is the percentage change formula, which states that to find the percent change between two values you need to take the new value represented by V2 subtract it by the old value or V1, then divide the result by the absolute value of the old value and finally multiply the output by one hundred to get the value for percent change. The right formula represents the numerical change or difference between two values, which uses subtraction to take the difference between new and old to get the basic mathematical difference in values. These differences for all features were calculated within excel and can be found within “ObjectiveData.csv” on the projects GitHub repo. When observing the difference values across all the MIP players, two clear trends were present. The first was that for all these players their Points Per Game between the previous and current season increased by a minimum of 20% or more. The second signal was related to Win Shares (WS), all these MIP candidates saw an increase of 2 plus. These two observed trends came to serve as the ground truth for this project and defined the conditions that a player needs to fulfill in order to be classified as a breakout player. Following the advice of my fellow colleges, the decision to examine the potential for threshold ranges for these critical features was implemented using confidence intervals. The dataset used to calculate these confidence intervals for PTS and WS can be found on GitHub. The results of these two confidence intervals indicated that the range in values for these two features ranged significantly higher than the values being used to test the hypothesis. Ultimately, the decision to stick to the previously observed PTS and WS difference values was made due to the negative impact these value ranges had on the class imbalance between breakout and non-breakout players.

With the necessary background information and analysis provided, it is now time to demonstrate how the target breakout feature was created within the notebook. The goal was to calculate each player’s season to season percentage difference in PTS and numerical difference in WS using “.shift(-1)” method was essential in achieving the desired difference values. Method documentation can be found at the end of the report.

Player	Age	Season	PTS	<u>next_PTS</u>
AJ Green	23	2022	3.3	4.5
AJ Green	24	2023	4.5	7.4
AJ Green	25	2024	7.4	10.4



Above is a figure that is helpful to conceptually understanding how the shift method works. The shift method examines the column it is attached to, and depending on the set value it will shift the values either up with positive values or down when given negative values depending on that value input into the brackets. Using shift(-1) allows for AJ Green's future PTS values to appear in the previous or current seasonal year row, which is needed to calculate the difference in values and can be seen by the blue arrows. Two new columns next_PTS and next_WS were created to hold the shifted future seasonal values. Due to their being NBA players within the observations not playing across all 11 seasons, along with the most recent season 2025 not having any future data to pull from, the decision to remove the records that contained null values was made. After dropping the rows with null values, the number of records lost was roughly 1,600 and a loss of just over 400 unique NBA players was removed.

With each of the rows within the dataset having two seasons worth of data for the desired features, the next step was to calculate the percentage and actual differential values using the formulas presented in the above figure. Something important to highlight here is the way the percentage difference between PTS and next_PTS is calculated within the "Breakout_Final_Notebook" on GitHub. Rather than the value being formatted like 80%, the decimal equivalent value 0.80 was the desired result removing the need for multiplication by 100 and is replaced by subtracting by one. The calculated percent difference for PTS was stored in a new column called PTS_difference with the calculated difference in WS results being stored in the column WS_difference.

With both PTS_difference and WS_difference for every player calculated all that was left was to use the two columns to create the binary target variable. In order for a player to be labeled as breakout or the value of one they needed to have a PTS_difference value greater than or equal to 0.20 and WS_difference values greater than or equal to 2. Failure to meet either of these conditions caused a player to not be labeled as breakout or zero. The final totals for each group in the breakout target feature was 4084 records for non-breakout and 313 observations for

players classified as breakouts or 1. With the target variable now created, the project was able to transition into model development and implementation. During the early stages of this project, feature selection using ANOVA-F was experimented with but ultimately was not implemented in the final notebook due to worse model performance observed when directly limiting the total number of features within the dataset.

Methodology Setup and Evaluation

As mentioned earlier in the report, a detailed explanation for the train, validation, and test splits used for the project's machine learning models is needed. During previous project updates, a traditional train/test split along with cross_validate was used on the 4 project models chosen. The problem using traditional data splitting and cross validation is that with random folds under this structure the model can train on the 2021 season and be validating on 2020 season. The fundamental problem with this is that the model is training on a future season, resulting in the models training on the answers making it unfair or basically cheating. In addition, the inconsistency and random nature of seasons that can be shuffled into the training folds negatively impacts each model's ability to interpret and predict breakout. The solution to this problem is utilizing a season-based or time series train, validation, and test data split.



The above figure was created with the help of AI and provides a visual example of the seasonal train and validation system. The most recent season in the dataset is the 2024 season which was set aside for the final testing which is done after the model has been fully trained, validated, and model parameters have been tuned. The remaining seasons are used to train each of the machine learning models sequentially. Starting four seasons in the model in the first fold trains on the 2014 to 2017 data using the 2018 season to validate the performance. Then the model loops to the next season, sliding the previous validation season into the training data and the following season becomes the validation for the second cross validation fold. This process of

iteratively training and validating each season ensures that the models follow the correct order each season occurs within the dataset allowing the machine learning models to capture the effects of the time series present within the project's dataset.

Random seed throughout all 3 of the machine learning models is set to be 42. The following programming libraries were used within the code for this project: numpy, pandas, matplotlib.pyplot, seaborn, from scipy.stats (t/wilcoxon/ttest_rel/norm), time, train_test_split, feature_selection, DummyClassifier, LogisticRegression, XGBClassifier, and from sklearn.metrics (accuracy_score/ precision_score/ recall_score/ f1_score/ confusion_matrix/ classification_report). In this next section detailed descriptions will be provided for the final chosen algorithms for this project are naive majority DummyClassifier, Baseline LogisticRegression, tuned LogisticRegression, and a tuned XGBoost. Documentation links used for each of these models can be found in the citations section of the report.

The first of the three algorithms naive majority DummyClassifier searches for the class with the most values and always predicts that class. The algorithm does not attempt to learn any patterns or attempt any predictions towards the minority class, which in the case of this project is breakout. This algorithm is designed to serve as a fundamental benchmark for the other algorithms' performance in predicting the target minority class breakout. LogisticRegression model calculates a weight or impact value for every feature provided by the dataset to create score values.

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

LogisticRegression utilizes the sigmoid function, formula can be seen in the figure above, to convert each of these feature score values into probabilities that fall between the range of zero to one. The scores are represented by z within the sigmoid function equation. The combined probability is then used to predict whether the player is a part of the breakout label. LogisticRegression also utilizes the loss function to penalize misclassifications and learn from both successful and unsuccessful predictions.

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Above is the formula or mathematical equation for the loss function with the actual value being represented by y and the predicted value being equal to p . The hyperparameters for

LogisticRegression were tuned manually and there were a total of 5 parameters that were adjusted to improve prediction performance. Below is a figure showing the parameters used to achieve the best performance within the final tuned LogisticRegression model.

```
model3 = LogisticRegression(  
    penalty="elasticnet",  
    l1_ratio=0.25,  
    solver="saga",  
    C=0.10,  
    class_weight="balanced",  
    max_iter=10000,  
    random_state=42
```

The penalty parameter specifies the regularization impact on the loss function. The four options for this parameter are L1 or lasso, L2 or ridge, elasticnet, and none. Penalty set to elasticnet achieved the highest evaluation metric performance, which indicates the complexity present within the data since elasticnet is attempting to find the balance between L1 and L2 ratio on regularization. L1_ratio is a parameter that only can be used on LogisticRegression when the penalty parameter is set to be elasticnet. When l1_ratio is set to be equal to zero, the model becomes pure L2 and when the l1_ratio is set to one, the model becomes strictly L1 regularization. Any value in-between zero and one for l1_ratio becomes a balance between the two regularization terms with the tuned model performing the best when the value is set to 0.25. The third parameter was solver, which is used for optimization of the problem loss. Solver has six unique options which are lbfgs, liblinear, newton-cg, newton-cholesky, sag, and saga. Each solver works differently depending on the objective or penalty the solver works with. Since the best performing tuned model uses elasticnet, saga is the only solver that works with elasticnet. Parameter C represents the inverse of regularization strength impacting the model's penalty. When C is set to a large value like 100, the strength of regularization is lowered resulting in the loss function penalization to be weaker. When C is set to small values like 0.01, the regularization strength inversely increases significantly making the loss penalty more severe within the loss function. The final parameter is class_weight which adjusts the weights of each class from their default. Setting the class_weight to balanced increases the penalty on the loss function for misclassifying the target breakout, forcing the LogisticRegression model to make more predictions for breakout class 1. The last algorithm is XGBoost which uses decision trees to learn iteratively or sequentially from each additional tree run. The goal of each tree is to correct and improve prediction errors made by the previous tree. After all the decision trees are run, their outputs get combined in a probability to make its breakout prediction.

```
model4 = XGBClassifier(  
    objective='binary:logistic',  
    eval_metric='logloss',  
    scale_pos_weight=scale,  
    n_estimators=50,  
    max_depth=7,  
    learning_rate=0.01,  
    random_state=42  
)
```

Similar to LogisticRegression, the hyperparameters for the tuned XGBoost model were done manually and there are three parameters that are worth discussing. Parameter `n_estimators` decides or sets the number of trees or rounds for the gradient boosting. Setting the value of `n_estimators` to a large value like 10000 significantly increases the run time for training the XGBoost model. The second parameter `max_depth` increases quite literally the depth of layers for each individual tree. Increasing the value of `max_depth` allows each individual tree to consider more features and make predictions for the target class considering more information and interactions between complex dataset features. The `learning_rate` parameter decides the impact each decision tree has on the final prediction made on the target class.

Now that every machine learning algorithm has been provided, a detailed explanation into how they work the evaluation metrics used for comparing their prediction performance is next. All project algorithms are evaluated by the same five metrics, which are confusion matrix, accuracy, precision, recall, and f1-score. Confusion matrix is a grid visual that shows the distribution of correct and incorrect predictions made for each class. In the context of this project the confusion matrix is a two-by-two grid matrix containing four value counts: true positives (TP) which are the number of correct target class 1 predictions, false positives (FP) which are the count of incorrect class 1 predictions, true negatives (TN) which are the number of correct predictions made for class 0 or non-breakout, and finally false negatives (FN) which are the amount of incorrect class 0 predictions made by the model. In an ideal world, each algorithm would desire their confusion matrix values to be in either TP and FP with as few misclassified predictions like TN or FN as possible.

$$\text{Accuracy} = (TP + TN) / \text{All}$$

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$F = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

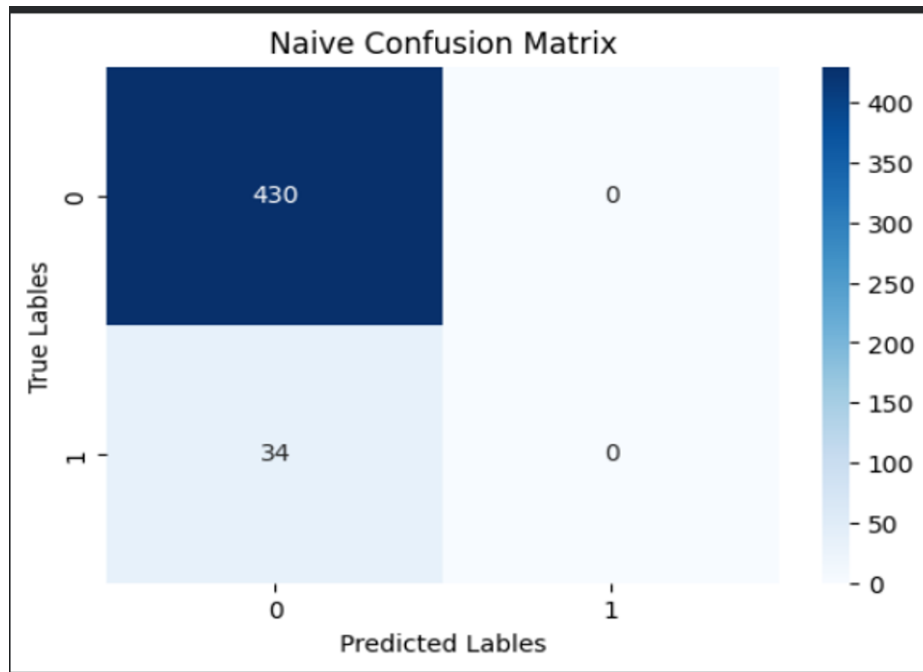
The above figure contains the formulas for the remaining four evaluation metrics that will be discussed. The next metric accuracy represents the total number of correct predictions percentage. Precision calculates how many predictions made for the target class 1 were actually part of class 1. Recall measures the percentage of class 1 predictions actually identified. The last evaluation metric f1-score represents the harmonic mean between precision and recall, which means that a model can only achieve a high f1-score if both precision along with recall are high values as well. Two statistical comparison methods were chosen to evaluate the performance of the best performing model. Paired t-test compares the means between two models using the validation folds. This test is essential for this project to analyze whether the project's best performing model can perform significantly better than baseline and alternative models. The McNemar's test is fundamentally great for the goal of this project due to it wanting to compare two classifiers on the same data to evaluate the degree of disagreement between algorithms. Verifying that the best performing prediction model has a different rate of errors in comparison to a baseline model proves whether the model has a significant difference in the number of errors.

Discussion of Results

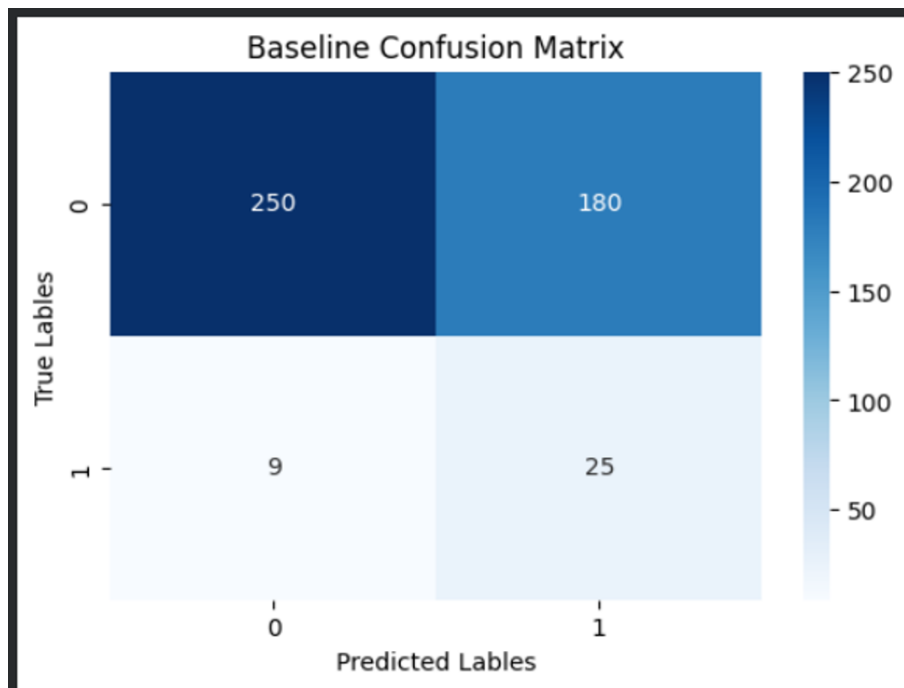
As discussed in the methodology section of the report, there were a total of 3 algorithms used to classify and predict breakout NBA players. However, the decision to run four total models was made in order to gauge model performance better. The four models that will be compared are the naive majority, Baseline Logistic Regression, a tuned Logistic Regression, and the tuned XGBoost models. Below is a figure that provides a table that contains the evaluation of metric scores or values along with train and test run times for all four models.

Comparison	Accuracy	Precision	Recall	F1-score	Training Time	Testing Time
Naïve Majority	0.927	0.000	0.000	0.000	0.0010	0.0002
Baseline Logistic	0.593	0.122	0.735	0.209	0.7835	0.0016
Tunned Logistic	0.601	0.132	0.794	0.226	10.0607	0.0020
Tunned XGBoost	0.711	0.148	0.618	0.239	0.2910	0.0049

The naive majority had the highest accuracy among all the models, but the cost of that accuracy is the zero values across the board for precision, recall, and f1-score showing that the model does not make a single prediction for the target class. Baseline Logistic Regression sacrificed accuracy, but in exchange provides significant improvement from zeros across the board for the 3 other already mentioned evaluation metrics. The tuned Logistic Regression model almost takes ten times as long to run in comparison to its baseline counterpart. In return, the tuned Logistic sees a 0.01 increase to precision, 0.059 increase to recall, and 0.017 increase to f1-score. The final model run was tuned XGBoost turned out to be the best sacrificing recall score in exchange for improved accuracy, precision, f1-score, and training speed. With f1-score along with precision being the most important metrics to evaluate breakout classification and prediction, tuned XGBoost was clearly the best performing model for the project. The final evaluation metric that needs to be compared between the four models are the confusion matrixes.

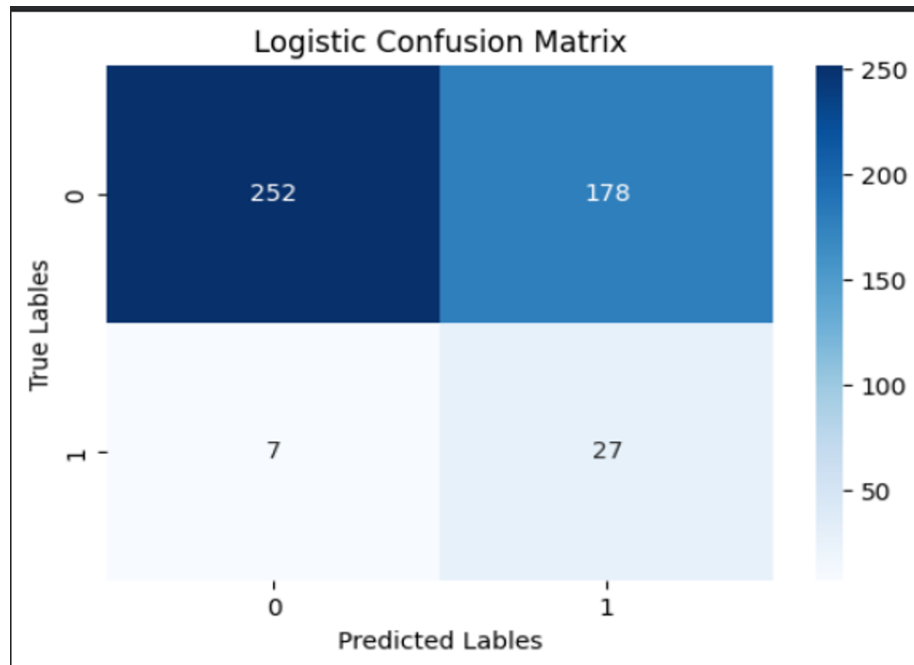


Looking at the naive majority confusion matrix, it is clear to see that the model only made predictions for class zero. The class imbalance highlights that even when the model ignores the target breakout or class 1, the model is still able to achieve a 93% accuracy by just predicting all players as being class 0.

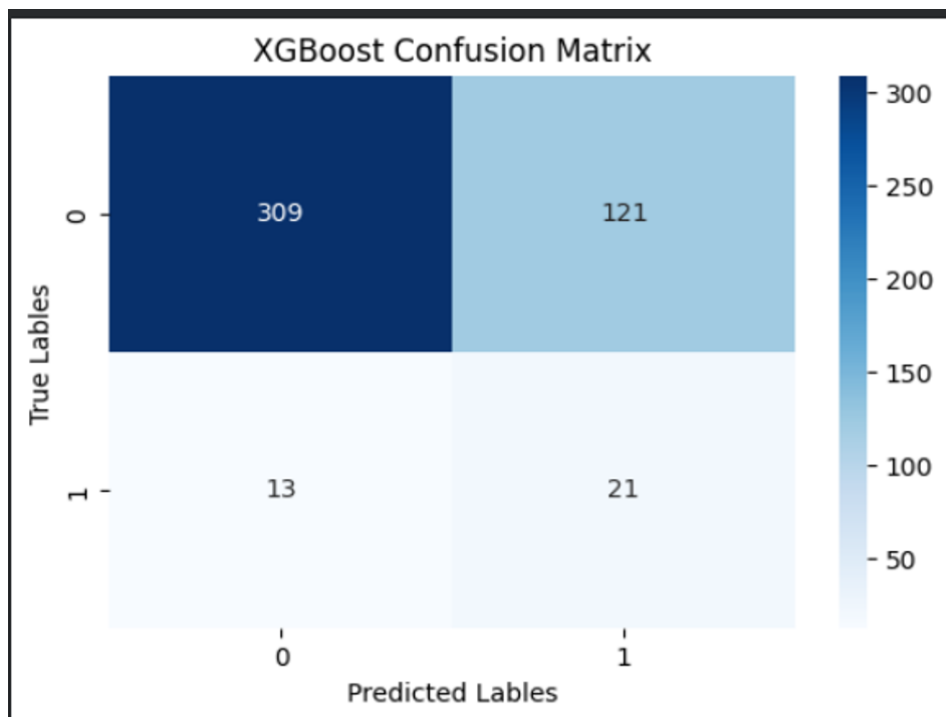


The baseline confusion matrix shows the effect of `class_weight = balanced` impact on the loss function penalizing not predicting class 1 more. However, this also creates an overcorrect

effect which can be seen with the number of incorrectly predicted class 1 values or FP being equal to 180.



This tuned Logistic confusion matrix shows minor changes in comparison to its non-tuned counterpart. Two false positive predictions were lost, and true positives gained two predicts signaling improved classification for class 1. The same pattern occurs with class 0 were two false negatives lost and two true negatives gained and can be seen within the above matrix.



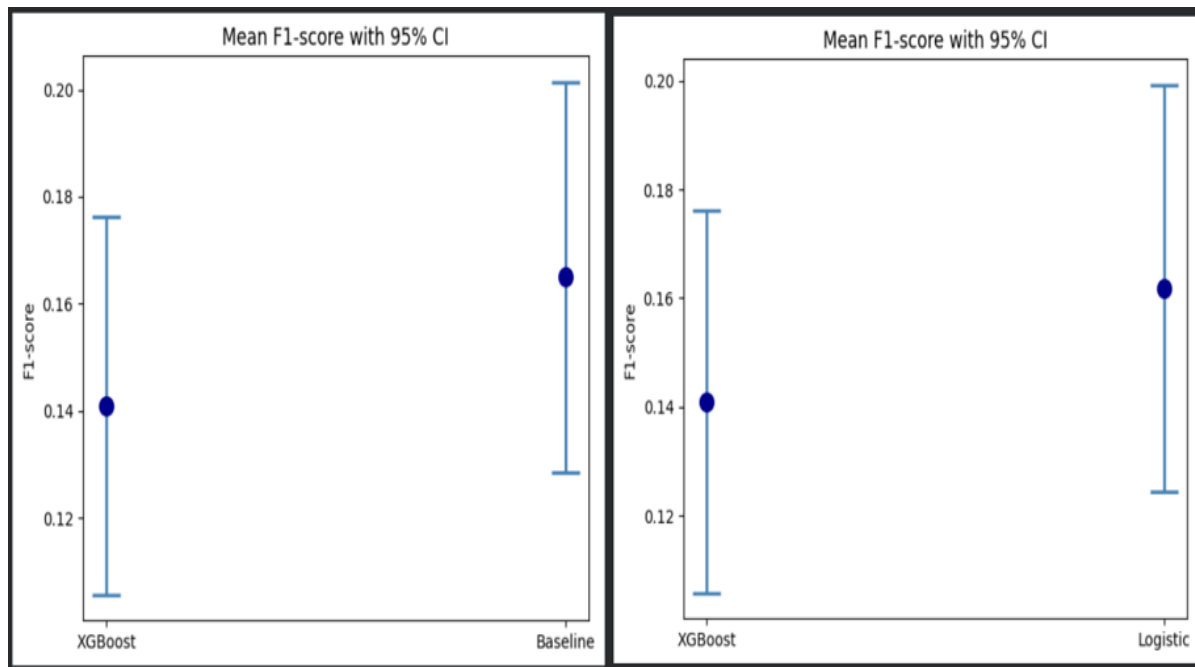
The tuned XGBoost confusion matrix shows significant improvement in classifying class 0 as there is a significant increase in true negative count by 57 and decrease in false positives by the same amount 57. However, class 1's correct predictions decreased and the number of misclassified class 0 increased, which harmed potential for growth in model predictive power.

After comparing all four model's performance to classifying and predicting class 1 and deciding XGBoost was the best model, the need to determine statistical significance was the next step. The below figure contains the results from the Paired t-test and McNemar's test metrics and p-values for the two comparisons made against the XGBoost model.

	Baseline vs XGBoost	Logistic vs XGBoost
Paired t-statistic	-1.7391	-1.6546
Paired p-value	0.1569	0.1733
Paired significance	Fail to Reject H0	Fail to Reject H0
McNemar Chi-squared	21.9248	19.3798
McNemar p-value	0.000003	0.000011
McNemar significance	Reject H0	Reject H0

The goal was to compare the tuned XGBoost model against the Baseline Logistic Regression model and compare the tuned XGBoost model against the alternative tuned Logistic Regression model to determine if the XGBoost model is statically significant. Starting with the Paired t-test results for both comparisons using the validation scores for all three models, the baseline comparison t-test resulted in a p-value of 0.1569, which was a value significantly larger than the alpha of 0.05 not providing enough evidence to reject the null hypothesis and prove that the XGBoost results are not statistically different or significant. However, it does beat the Baseline Logistic model in terms of evaluation metric performance, which can be seen in the figure at the beginning of this section. However, the McNemar's test between the two models gave a small p-value above which justified the rejection of the null hypothesis concluding that the Baseline Logistic and XGBoost has significantly different errors rates. The tuned Logistic Regression versus XGBoost model showed almost identical results for both statistical tests. The Paired t-test p-value was too large, leading to the conclusion that we cannot reject the null hypothesis since there is a lack of statically significant difference between models. This means that XGBoost while outperforming tuned Logistic Regression in evaluation metric performance

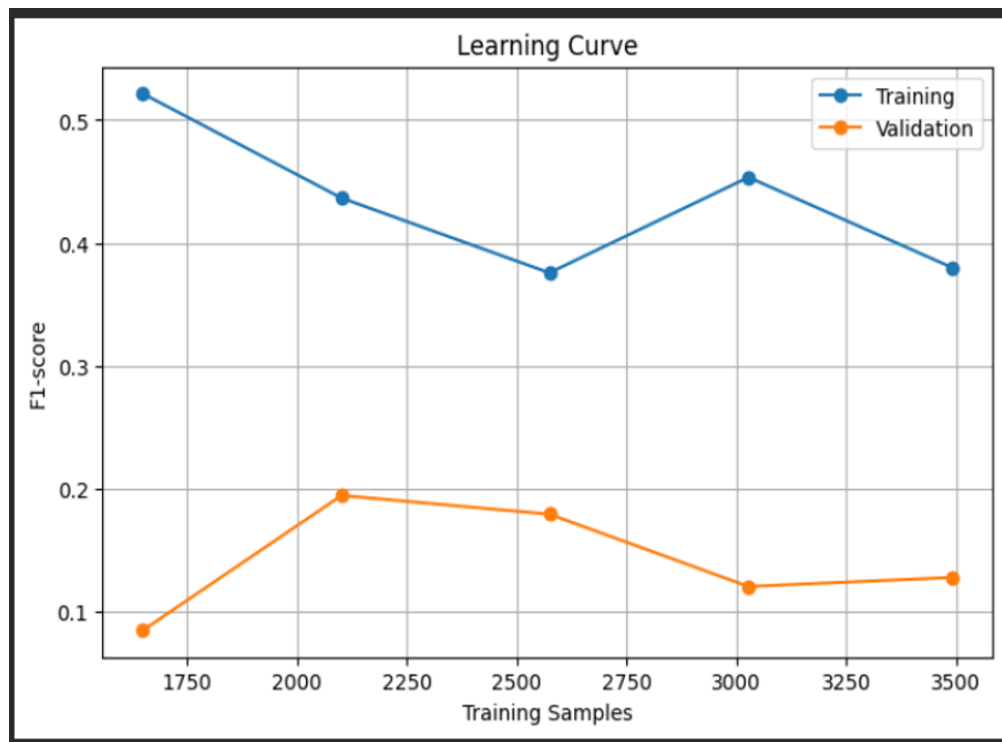
is not statistically significant. The McNemar's test small p-value seen in the figure above provided evidence to conclude the error rates between XGBoost and tuned Logistic Regression were significantly different.



The above figure contains the confidence intervals for both model comparisons calculated using the mean and standard deviation for their model validation results. As can be observed, the two separate model comparisons have almost identical confidence intervals all the way down to the ranges of values.

A brief error analysis was conducted to attempt to understand trends and patterns that occur within XGBoost's misclassified predictions. An observable trend when looking at the mean statistics for both false positives and false negatives was that both had young age averages as well as low average minutes played per game (MPAVG). This signals that XGBoost model struggled with identifying and predicting which group these young players with minimum playing time belong to.

Gathering all the evaluation metrics and statistical analysis results from the XGBoost model performance, the project can now determine if our project hypothesis can be accepted or rejected. Due to the low values for f1-score and precision values along with being unable to claim statistical significance against alternative models' ability to predict breakout players. The project hypothesis is refuted as the model is unable to meaningfully prove that it has noteworthy predictive power that NBA players will breakout.



The above figure is a learning curve between the training and validation data for the XGBoost model. The gap between the blue and orange line signifies the model is overfitting the training data, causing worse performance when applied to the validation data. This also indicates that adding more season's worth of NBA data to the model won't improve overall performance.

The methodology of this project benefited from seasonality and analyzing the data through time series validation. While not directly discussed in this report, the machine learning models in the final notebook performed overwhelmingly better in terms of evaluation of metrics scores compared to the models using traditional train/test splits and cross validation methods. The previous evaluation metric scores from past project updates can be found on the project GitHub. The biggest limitation for this project was the difficulty handling the severe class imbalance within the target feature. Even using model parameters to try and balance the data, every model still suffered in terms of predictive performance because of the struggle to properly handle the imbalance. The project was unable to determine if bias was present within the dataset or if it impacted data collection, labeling, sampling, and potential performance across subgroups. This model would not have much overall impact on the NBA landscape due to the lack of statistical significance and XGBoosts predictive performance. This model if used would likely cause more harm than good due to the inconsistency and inability to classify the breakout players, which was the entire goal of the project and models. There are three takeaway lessons that were learned from the completion of this project. The first is that professional sports data is difficult to work with and analyze due to the amount of multicollinearity present between the majority of player statistics. The second takeaway was that the amount of time spent conducting research for this project took more time than initially expected. Approximately 75% of the

project timeframe, around one month was spent working on this project and spent researching evidence and machine learning models' details with the minimum amount of time spent modeling was overall surprising. The final lesson from this project was to spend more time exploring the dataset during exploratory data analysis.

Project Conclusion

This project started with passion for NBA basketball and wanted to see if it was possible to accurately classify and predict players' ability to breakout. The hypothesis for the project is that NBA players that see an increase of 20% or more to their Points Per Game (PTS) and an increase of 2 or more Win Shares (WS) from one season to the next will breakout. Using statistical analysis and evaluating the performance of the best model XGBoost, the project hypothesis is unsupported lacking statistical significance and overall predictive power. Future work would involve redefining how to define a breakout NBA player as well as potentially using additional data sources to provide new features for machine learning models to use. This report was written fully by the human author and did not use AI in any way to write any words directly found within the report. AI was used to evaluate project ideas throughout all stages, review machine learning models concepts, create certain code, and help debug coding issues.

Citations and References

Project Source Data Website: <https://www.basketball-reference.com/>

Project GitHub Link: <https://github.com/ceunson15/MSDS640-coursework>

Shift() Python Dataframe Documentation: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.shift.html>

DummyClassifier Scikit Documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.dummy.DummyClassifier.html>

Logistic Regression Scikit Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

XGBoost Documentation: <https://xgboost.readthedocs.io/en/stable/parameter.html>

XGBoost Parameter Information: <https://www.geeksforgeeks.org/machine-learning/xgbclassifier/>